Smart MCQ Solver Challenge

1. Abstract Summary

The Smart MCQ Solver Challenge requires predicting the three most probable answer options for each five-option question. The competition metric is Mean Average Precision at 3 (MAP@3), which rewards placing the correct option near the front of the ranked prediction. Three qualitatively different approaches were compared: a fine-tuned DeBERTa-v3-small multiple-choice transformer, a frozen Sentence-BERT (SBERT) embedding model with a neural classifier, and a from-scratch Bi-LSTM with Bahdanau attention. A weighted Borda ensemble of all three was also evaluated.

On the held-out validation set, DeBERTa achieved MAP@3 = 0.9825, SBERT+classifier 0.8008, Bi-LSTM 0.6392, and the weighted ensemble 0.9417. The final submission.csv uses DeBERTa's predictions, as it was the strongest standalone system and scored 0.74605 (rank 942) on the Kaggle public leaderboard.

2. Introduction

2.1 Problem Statement

The dataset contains 2,000 labelled training questions and 500 test questions. Each row has a prompt and five candidate answers (A–E); training rows also contain the correct label. Given a question and its five options, the task is to output a ranked top-3 subset most likely to contain the correct answer, scored by MAP@3.

2.2 Project Objective

The objective was to develop three qualitatively different modelling approaches spanning a from-scratch recurrent architecture, a pretrained-embedding hybrid, and a fine-tuned transformer; compare their ranking performance under a shared evaluation protocol; track all experiments in Weights & Biases; and produce a valid, well-formatted top-3 submission for the Kaggle competition.

2.3 Report Structure

The remainder of this report is organized as follows. Section 3 describes the dataset, exploratory data analysis and preprocessing. Section 4 discusses the tokenization strategies used across the three models. Section 5 presents the modelling pipeline together with the three model architectures, their fine-tuning strategies, and the ensemble. Section 6 evaluates and compares model performance using validation metrics, training curves, computational trade-offs, Kaggle results, and error analysis. Section 7 discusses key learnings, challenges faced, methodological limitations, and future work. Section 8 lists references.

3. Dataset & Preprocessing

3.1 Dataset Description

Property	Value	Implementation
Train	2,000	train.csv
Test	500	test.csv
Train / Val split	1,800 / 200	90/10 stratified, seed=42
Options	A–E	5 choices
Metric	MAP@3	custom metric function

Each training row contains the question prompt, five candidate answer fields (A–E), and the correct answer label, while the test rows contain the prompt and five candidate answers without the target label.

3.2 Exploratory Data Analysis

EDA found 242 duplicated prompt strings. Answer labels were mildly imbalanced: A=369, B=490, C=459, D=358, E=324. Mean prompt length was about 18.2 words.

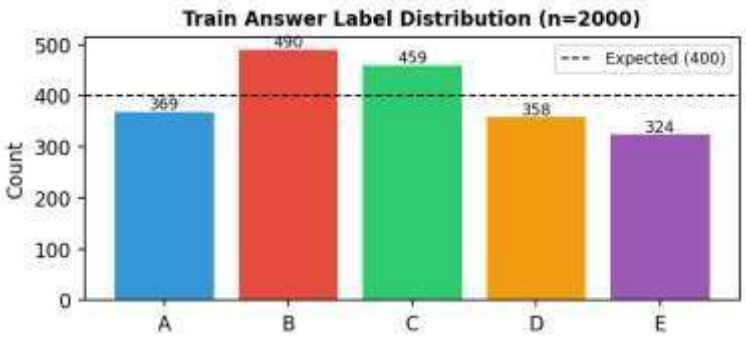


Figure 1. Training-set answer-label distribution.

3.3 Data Preprocessing

The Bi-LSTM uses lower-cased, whitespace-normalized text with non-alphanumeric noise removed while retaining sentence punctuation. SBERT and DeBERTa use the original prompt/option strings and their own internal encoding pipelines.

4. Tokenization Strategy

4.1 Primary Tokenizer

The best-performing model uses the Hugging Face tokenizer associated with microsoft/deberta-v3-small, loaded through AutoTokenizer. This is a SentencePiece-based subword tokenizer paired with DeBERTa-v3's disentangled-attention encoder, chosen because it handles out-ofvocabulary and rare terms gracefully while keeping sequence lengths compact for the 128-token budget used here.

4.2 Implementation Details

The notebook constructs five prompt–option pairs per question, tokenizes them in batches, truncates to 128 tokens, and pads to a fixed length before passing them through a custom DataCollatorForMultipleChoice, which reshapes the flattened (question × option) batch back into the [batch, 5, seq_len] tensor the multiple-choice head expects.

4.3 Experimental Comparison

The Bi-LSTM instead uses a custom word-level vocabulary (3,719 tokens) and an explicit <SEP> token, since it has no pretrained subword model to draw on. SBERT uses the WordPiece tokenizer internal to its SentenceTransformer package. The final choice of DeBERTa's tokenizer for the primary model follows directly from choosing DeBERTa as the primary architecture (Section 5.1); the comparison across all three models therefore spans learned word-level sequences, frozen sentence embeddings, and jointly fine-tuned contextual transformer inputs.

5. Modeling & Experimentation



Figure 2. Overall pipeline implemented.

After EDA and a stratified split, each model applies its own representation pipeline as described in Section 4. All systems rank the five options and return the top three; the four rows below summarize each system's core configuration before the detailed subsections.

Model	Optimizer / LR	Schedule	Epochs	Batch
5.1 DeBERTa-v3-small	AdamW; 2e-5	Linear, 10% warmup	5	8
5.2 SBERT + classifier	AdamW; 1e-3	OneCycleLR	100	64
5.3 Bi-LSTM + attention	AdamW; 5e-4	CosineAnnealingWarmRestarts	30	32

5.1 Model 1 — Fine-Tuned Transformer: DeBERTa-v3-small

microsoft/deberta-v3-small is loaded with AutoModelForMultipleChoice. Each question becomes five (prompt, option) sentence pairs; AutoTokenizer applies truncation and max_length=128 with fixed-length padding, and the multiple-choice head produces one logit per option, which is ranked to obtain the top three.

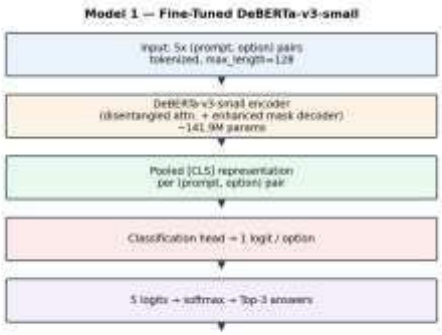


Figure 3. Fine-tuned DeBERTa-v3-small multiple-choice pipeline.

Salient architectural features. DeBERTa extends the Transformer encoder architecture through disentangled attention, which separately represents content and positional information during attention computation[1]. It also uses an enhanced mask decoder as part of its pretraining design. These characteristics provide strong contextual representations, which are useful for evaluating the semantic and contextual compatibility between a question and each candidate answer. The model was selected as the primary transformer because the task requires exactly this kind of contextual prompt–option relevance judgement; unlike the SBERT hybrid, it can fine-tune the contextual representation directly for the MCQ decision rather than relying on a fixed embedding summary.

Fine-tuning strategy. The encoder and classification head are optimized end-to-end with AdamW at a peak learning rate of 2e-5, a linear decay schedule with a short warmup phase (22 steps), 5 epochs, batch size 8, and FP16 mixed precision. No DeBERTa encoder layers were frozen — the pretrained encoder and multiple-choice classification head were fine-tuned end-to-end; no quantization was applied. FP16 mixed-precision training was used only to reduce GPU memory usage during training, not as a form of post-training quantization.

5.2 Model 2 — Custom Deep Learning Model: SBERT + Neural Classifier

all-MiniLM-L6-v2 produces frozen 384-dimensional normalized embeddings for the prompt and each option. Per option, the notebook computes cosine similarity, L2 distance, Hadamard-product norm, and normalized cosine-similarity rank, yielding 20 features per question. A custom MLP classifier maps these features to five logits.

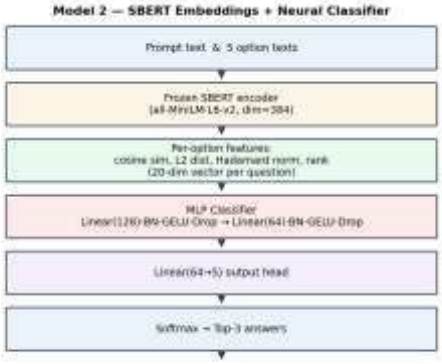


Figure 4. SBERT feature pipeline.

This model represents a middle ground between the fully-scratch Bi-LSTM and the fully fine-tuned DeBERTa: it reuses a frozen pretrained sentence encoder for its language understanding, but the classification logic itself — the similarity-feature engineering and the MLP head — is custom-built and trained from scratch for this task. Only the downstream classifier (11.6K parameters) is trained with AdamW (lr=1e-3, OneCycleLR) for 100 epochs, batch size 64; the SBERT encoder itself remains frozen throughout.

5.3 Model 3 — RNN-Based Model: Bi-LSTM + Bahdanau Attention

Five sequences are formed as [prompt] + <SEP> + [option]. The custom word-level vocabulary contains 3,719 tokens. Tokens enter a randomly initialized 128-dimensional embedding, then a two-layer bidirectional LSTM (hidden size 256 per direction; 512-dimensional output), followed by Bahdanau additive attention, LayerNorm/dropout, and a shared linear option scorer.

A sequence-based model is a reasonable choice for this task because answer options must be evaluated relative to an ordered prompt, and recurrent processing lets the model accumulate context word-by-word rather than treating the prompt as a bag of tokens. Bidirectionality was used so that each token's representation is informed by both preceding and following context, which is useful for prompts where the deciding evidence for an option can appear after the token that seems most relevant on a first pass. Bahdanau attention was added instead of relying on the LSTM's final hidden state alone because MCQ prompts are often longer than the option text, so a single fixed-size summary vector tends to lose the specific span relevant to a given option; attention lets the model re-weight each token per (prompt, option) pair instead of compressing the whole prompt once.

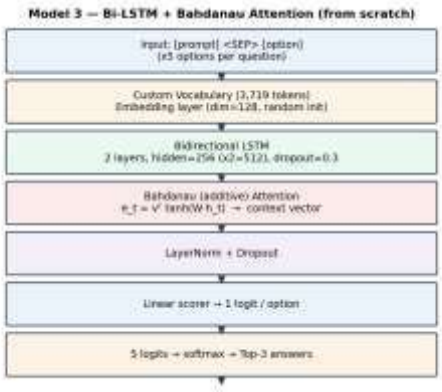


Figure 5. Bi-LSTM + Bahdanau attention architecture.

This model is trained fully from scratch with AdamW (lr=5e-4, CosineAnnealingWarmRestarts, T0=10, T_mult=2) for 30 epochs, batch size 32; no layers are frozen. It exists primarily as a from-scratch lower bound: with no pretrained word representations, it should be expected to lag SBERT and DeBERTa, and the size of that gap (Section 6.3) is itself evidence for how much pretraining contributes to this task.

5.4 Weighted Borda Ensemble

The three models' ranked top-3 predictions are combined with a weighted Borda count: each model casts votes for its ranked options (3 points for 1st choice, 2 for 2nd, 1 for 3rd), scaled by that model's own validation MAP@3, and the five options are re-ranked by their summed weighted score to produce the ensemble's top-3. This lets the stronger models dominate the vote while still letting weaker models contribute on questions where they agree. The ensemble is logged as its own W&B run and evaluated in Section 6.3; because DeBERTa alone already outperforms the ensemble on validation, DeBERTa's own predictions are used for the final Kaggle submission rather than the ensemble.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

MAP@3 is the competition metric. With one correct label, AP@3 is 1.0 if the correct option is ranked first, 0.5 if second, 0.333… if third, and 0 if absent from the top three. Accuracy@1 checks the top-ranked option; Accuracy@3 checks whether the correct option appears in the top three; macro-F1 evaluates top-1 predictions with equal weight for the five classes. All values below are validation results on the 200-row stratified validation set.

6.2 Training Performance

All three models log per-epoch metrics (train loss, MAP@3, Accuracy@1/3, macro-F1) directly to Weights & Biases during training — BiLSTM and SBERT-classifier via explicit wandb.log() calls inside the training loop, DeBERTa via Hugging Face Trainer(report_to="wandb"). The training metrics were logged to the corresponding W&B runs throughout training, and the training curves shown in Figures 6–9 were generated from these recorded per-epoch metrics.

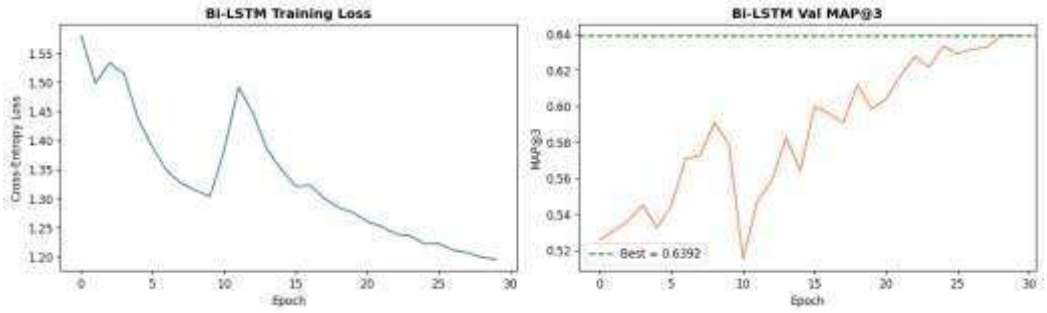


Figure 6. Bi-LSTM training loss and validation MAP@3 scores.

Bi-LSTM loss decreases overall while validation MAP@3 rises to 0.6392 by epoch 30. The model remains substantially behind the pretrained alternatives.

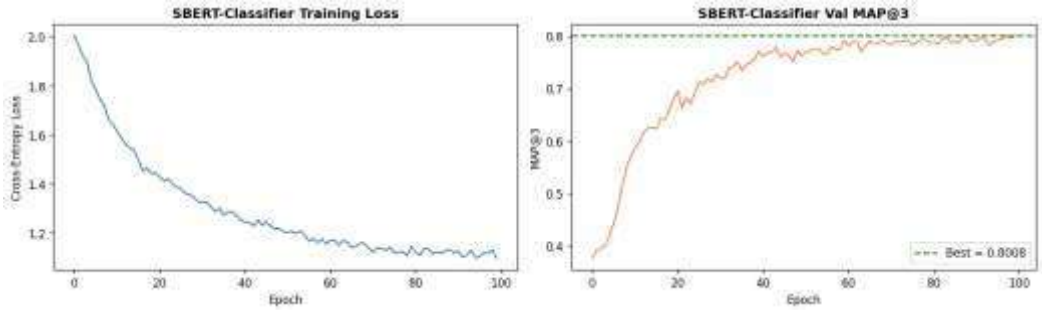


Figure 7. SBERT-classifier training loss and validation MAP@3 scores.

The SBERT classifier improves rapidly and reaches 0.8008 MAP@3. Its validation curve becomes relatively flat later in training, consistent with a small trainable classifier operating on fixed pretrained embeddings.

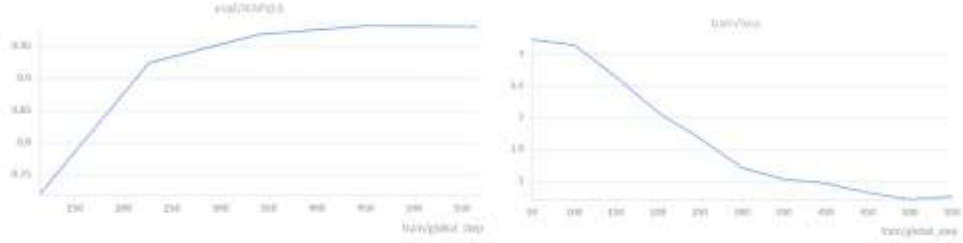


Figure 8. DeBERTa training loss and map@3 scores

DeBERTa's training loss falls sharply within the first ~150 steps and continues decreasing to below 0.8 by the end of the run, while the learning rate decays linearly from its 2e-5 peak per the warmup schedule described in Section 5. It also completed 5 epochs in approximately 6.5 minutes on the same T4 GPU and needs FP16 mixed precision to fit comfortably in memory. Evaluation-side metrics logged by the Trainer at each epoch checkpoint show MAP@3 reaching its best value of 0.9825 by the final epoch — a much faster convergence than Bi-LSTM or SBERT+classifier, both of which are still improving when their training budgets end, consistent with fine-tuning a model that already carries strong pretrained language representations into the task.

6.3 Comparative Results

Model	MAP@3	Accuracy@1	Accuracy@3	Macro-F1
DeBERTa	0.9825	0.9650	1.000	0.9641
Weighted Borda ensemble	0.9417	0.8950	0.9950	0.8932
SBERT + classifier	0.8008	0.6850	0.9500	0.6738
Bi-LSTM + attention	0.6392	0.4450	0.8700	0.4239

DeBERTa is the clear validation leader. The ensemble is strong but is pulled down by rankings from weaker models, so it is not used for the final submission.

6.4 Computational Trade-offs

Performance gains come at increasing computational cost. Bi-LSTM (3.1M parameters) and SBERT and classifier (11.6K trainable parameters) are cheap to train — the Bi-LSTM's 30 epochs and the classifier's 100 epochs each complete in minutes on a single T4 GPU. DeBERTa-v3small (141.9M parameters) needs FP16 mixed precision to fit comfortably in memory. In absolute terms DeBERTa's MAP@3 gain over BiLSTM and over SBERT and classifier is large enough to justify that cost for this competition, but the SBERT and classifier is the strongest result per parameter trained and would be the more practical choice under tight compute or latency budgets.

6.5 Kaggle Performance

Item	Result
Submission	submission.csv
Format validation	Passed: 3 distinct A–E predictions per row
Validation MAP@3	0.9825

Kaggle public score	0.74605
Kaggle rank	942

6.6 Error Analysis

6.6.1 Ranking behaviour

The weaker models show a large Accuracy@1–Accuracy@3 gap: Bi-LSTM 0.445 vs 0.870, and SBERT 0.685 vs 0.950. They often place the correct answer in the top three but have difficulty separating it from a plausible distractor.

6.6.2 Class imbalance

B is the most frequent training label (490) and E the least frequent (324). The lower macro-F1 of the weaker models is consistent with uneven class-wise performance, although the notebook does not establish a causal label-specific mechanism.

6.6.3 Architecture-level failure modes

- Bi-LSTM: learned word representations from the competition data provide limited semantic generalization compared with pretrained language models.
- SBERT classifier: four similarity statistics per option summarize the text too aggressively to model full prompt–option interactions, so a close distractor can outrank a correct but differently worded answer.
- DeBERTa: validation Accuracy@1=0.965 and Accuracy@3=1.000 indicate that remaining errors are primarily top-1 ranking errors rather than failure to retrieve the answer in the top three.

Question-level error analysis. The current analysis focuses on aggregate ranking behaviour because question-level validation predictions were not persisted alongside the original question text and gold labels.

7. Conclusion & Future Work

7.1 Key Learnings

The experiments show a clear performance progression from a scratch sequence model to a frozen semantic representation and finally to contextual fine-tuning. Bi-LSTM achieved 0.6392 MAP@3, SBERT+classifier 0.8008, DeBERTa 0.9825, and the weighted Borda ensemble 0.9417 on validation. The main conclusion is that jointly modelling each prompt–option pair with a pretrained contextual transformer is much more effective for this ranking task than learning word representations only from the competition data or reducing each option to fixed embedding-similarity statistics.

The difference between the local validation MAP@3 of 0.9825 and the public Kaggle score of 0.74605 also demonstrates that validation performance on a single split may not fully represent generalization to the hidden competition distribution — a useful reminder that a strong validation score is necessary but not sufficient evidence of real-world generalization.

7.2 Challenges Faced

Several implementation issues arose during development. Mixed-precision fine-tuning of DeBERTa initially failed due to an FP16/FP32 dtype mismatch, resolved by aligning the model and trainer precision settings. An early configuration used a longer max sequence length that caused out-of-memory errors on the T4 GPU; reducing MAX_LENGTH to 128 tokens resolved this without materially hurting MAP@3. Running inference on the unlabeled test set initially raised a KeyError inside DataCollatorForMultipleChoice, which expects a labels field; this was fixed by injecting dummy labels for the prediction pass only, which are discarded before generating the submission file.

7.3 Methodological Limitations

- Bi-LSTM vocabulary was built from train + validation + test text. No test labels were used, but this is transductive preprocessing and should be avoided for a strictly leakage-free benchmark.
- Models do not use identical preprocessing: Bi-LSTM uses cleaned fields; SBERT and DeBERTa use original strings.
- The notebook does not document a systematic multi-trial hyperparameter sweep.
- Question-level validation predictions are not persisted with question text, limiting reproducible concrete error analysis.

7.4 Future Work

- Ensemble DeBERTa with another strong, independently fine-tuned transformer.
- Evaluate imbalance-aware objectives (e.g. class-weighted loss) to target the E-class recall gap.
- Run a documented learning-rate / epoch sweep via W&B Sweeps.
- Export question-level validation predictions (with question text and gold labels) for concrete, example-level error categorization.

8. References

[1] He, P., Gao, J., & Chen, W. (2021). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543.

[2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.

[3] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.

[4] Hugging Face, Transformers Documentation.

[5] Sentence Transformers Documentation (all-MiniLM-L6-v2).

[6] PyTorch Documentation.

[7] scikit-learn Documentation.

[8] Weights & Biases Documentation, project 23f3004276-t22026.

[9] Kaggle, Smart MCQ Solver Challenge dataset and evaluation metric.